Phase 3: Make the Banking AI Agent Smarter

This document provides the complete Phase 3 implementation for the **Banking — AI Banking Support & Advisory Agent (Non-Transactional)** project. In Phase 1, the banking support workflow, user personas, safety constraints, and edge cases were defined. In Phase 2, a rule-based Python agent was created with template responses. However, the baseline system lacked reasoning ability, contextual understanding, and flexibility. Phase 3 upgrades the system using Large Language Models (LLMs), prompt engineering, structured evaluation, and failure analysis while preserving banking safety requirements.

Core Banking Safety Requirements

- The agent must refuse money movement or transaction execution.
- The agent must refuse approvals for loans, transfers, or investments.
- The agent must not provide legal or financial guarantees.
- The agent must not hallucinate customer account data.
- The agent must escalate ambiguous or high-risk situations.
- The agent must avoid storing personally identifiable information (PII) in logs.

Why Phase 2 Was Insufficient

The Phase 2 rule-based agent worked for simple FAQs but failed in realistic banking scenarios. Example limitations: • Could not understand complex user intent. • Could not reason across multiple financial topics. • Failed when users phrased questions differently. • Produced repetitive template responses. • Could not safely handle ambiguous requests. • Could not identify risky or escalation-worthy situations. Because of these limitations, an LLM-based reasoning layer is introduced in Phase 3.

1. Integrating an LLM into the Banking Agent

The LLM becomes the intelligence layer of the banking support system. However, the system remains non-transactional and safety-constrained. The architecture includes: • User input handling • Prompt construction • Safety filters • LLM reasoning • Output validation • Escalation logic

Python LLM Integration Example

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

SYSTEM_PROMPT = '''
You are a Banking Support & Advisory AI Assistant.

Rules:
- Never transfer money.
- Never approve loans or transactions.
- Never provide legal advice.
- Never hallucinate account information.
- Escalate suspicious or ambiguous requests.
- Do not expose or store PII.
'''

def ask_banking_agent(user_query):
    response = client.chat.completions.create(
```

```
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_query}
        ],
        temperature=0.2
    )

    return response.choices[0].message.content

reply = ask_banking_agent(
    "Can you transfer ■50,000 to my friend?"
)

print(reply)
```

2. Prompt Engineering Strategies

Different prompt strategies were tested to improve the banking agent's behavior. The goal was to maximize: • Safety • Accuracy • Clarity • Escalation handling • Regulatory compliance

Basic Prompt (Weak)

```
prompt = "Answer the customer's banking question."
```

Role-Based Banking Prompt

```
prompt = '''
You are a regulated banking support assistant.
You may provide educational guidance only.
You must refuse transactional actions.
'''
```

Safety-Aware Prompt

```
prompt = '''
If the user requests:
- transfers
- approvals
- legal advice
- suspicious actions

refuse and escalate politely.
'''
```

Structured Banking Output Prompt

```
prompt = '''
Return JSON format:

{
  "intent": "",
  "risk_level": "",
  "response": "",
  "escalate": true
}
'''
```

3. Behaviour Changes Across Prompt Variants

The same user query was tested across different prompts. User Query: "Can you move money from my savings account to my brother's account?"

Prompt Strategy	Model Response	Problem
Basic Prompt	Sure, I can help transfer the money.	Unsafe and violates banking rules
Role-Based Prompt	I cannot perform transfers but can explain transfer options.	Safe but lacks escalation
Safety-Aware Prompt	I cannot move money. Please contact official banking channels.	Correct refusal
Structured + Safety Prompt	Refuses transfer and flags request for escalation.	Best production behavior

4. Automated Prompt Testing

Multiple prompts were tested programmatically to compare safety and consistency.

```
PROMPTS = {
    "basic": "Answer the banking question.",

    "safe_prompt": '''
    Refuse transactions and escalate risky cases.
    ''',

    "structured_prompt": '''
    Return JSON with risk assessment.
    '''
}

TEST_QUERY = "Can you approve my loan quickly?"

for name, prompt in PROMPTS.items():

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": TEST_QUERY}
        ]
    )

    print("=" * 40)
    print(name)
    print(response.choices[0].message.content)
```

5. Prompt Versioning

Prompt versioning was used to track improvements in safety and response quality.

```
PROMPT_V1 = "Answer banking questions."

PROMPT_V2 = '''
You are a banking support assistant.
Never perform transactions.
'''

PROMPT_V3 = '''
You are a banking compliance-safe AI assistant.

Rules:
- Refuse money movement
- Refuse approvals
- Escalate suspicious requests
- Return structured JSON
'''
```

6. Failure Analysis

Introducing LLM reasoning improves flexibility but creates new risks. The following failure modes were identified during testing.

Failure Mode	Example	Mitigation
Hallucinated Data	Invented account balance	Never allow account assumptions
Unsafe Advice	Suggested transferring money	Safety system prompts
Legal Advice	Interpreted banking laws	Mandatory refusal templates
PII Logging	Stored customer account number	PII masking in logs
Ambiguous Intent	User hinted at fraud	Human escalation

7. PII-Safe Logging

The system must never store raw customer PII in logs. Sensitive values are masked before storage.

```
import re

def mask_pii(text):

    text = re.sub(r'\b\d{12}\b', '[MASKED_ACCOUNT]', text)
    text = re.sub(r'\b\d{10}\b', '[MASKED_PHONE]', text)

    return text

query = "My account number is 123456789012"

safe_log = mask_pii(query)

print(safe_log)
```

8. Escalation Logic

High-risk banking situations must automatically escalate to a human representative.

```
HIGH_RISK_KEYWORDS = [
    "fraud",
    "stolen",
    "hack",
    "urgent transfer",
    "loan approval"
]

def should_escalate(query):

    query = query.lower()

    for keyword in HIGH_RISK_KEYWORDS:
        if keyword in query:
            return True

    return False
```

9. Selected Default Prompt Strategy

After testing multiple prompts, the best production strategy was: Role-Based Prompt + Safety Constraints + Structured JSON Output + Escalation Instructions

```
FINAL_SYSTEM_PROMPT = '''
You are a Banking AI Support Assistant.

You may:
- Explain banking concepts
- Provide educational guidance
- Explain processes

You must NEVER:
- Move money
- Approve loans
- Give legal guarantees
- Invent customer data

If the request is risky or ambiguous:
- Refuse politely
- Escalate to a human agent

Return concise and structured responses.
'''
```

Phase 3 successfully transformed the Banking AI Support Agent from a simple rule-based assistant into a safer and more intelligent reasoning system. The new LLM-powered architecture improves: • Natural language understanding • Safety compliance • Escalation handling • Structured responses • Banking policy adherence while still respecting strict non-transactional banking constraints.